POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Ho Chi Minh City Campus — Faculty of Electronics Engineering 2

PRACTICAL LABORATORY GUIDE

INTEGRATING AI MODELS INTO 32-BIT MICROCONTROLLERS

CNN on ESP32: From Training to Deployment

Author: Dr. Nguyen Trong Kien

Email: kiennt@ptithcm.edu.vn | Phone: 0776789221

Ho Chi Minh City, March 2026

Table of Contents

Table of Contents 2
Part 1: Introduction to Neural Networks 4
1.1 What Is a Neural Network? 4
1.2 Simple Neural Networks versus Deep Neural Networks 5
1.3 How Training Works 6
Part 2: Convolutional Neural Networks (CNN) 7
2.1 The Scaling Problem with Fully Connected Networks 7
2.2 The Convolutional Layer 7
2.3 The Pooling Layer 8
2.4 The Fully Connected Layer and Flatten 9
2.5 Activation Functions 9
2.5.1 ReLU 10
2.5.2 Softmax 10
Part 3: Running CNN on Microcontrollers 12
3.1 The Case for Edge AI 12
3.2 How Models Are Stored on ESP32 12
Part 4: TensorFlow Lite for Microcontrollers 13
4.1 Overview of TensorFlow Lite 13
4.2 Optimization Techniques 13
4.3 The Deployment Pipeline 13
Part 5: Setting Up the Python Environment 14
5.1 Installing Dependencies 14
5.2 Building a CNN with Keras 14
Part 6: Lab 1 — Sine Wave Prediction with CNN1D 15
6.1 Objective 15
6.2 Architecture 15
6.3 Complete Code 15
6.4 Exercises 17
Part 7: Lab 2 — Gesture Recognition with CNN1D and IMU 19
7.1 Objective 19
7.2 Collecting Data 19
7.3 Preprocessing and Model 21

7.4 Exercises 24
Part 8: Lab 3 — Handwritten Digit Recognition with CNN2D 25
8.1 Objective 25
8.2 Model Architecture 25
8.3 Complete Code 26
8.4 TensorFlow versus TFLite Performance 27
8.5 Exercises 28
Part 9: Lab 4 — Converting to TFLite and Exporting for ESP32 30
9.1 Conversion 30
9.2 Verifying the TFLite Model 30
9.3 Exporting as a C Header 30
9.4 Exercises 31
Part 10: Lab 5 — Deploying the CNN on ESP32 32
10.1 Setting Up Arduino IDE 32
10.2 ESP32 Firmware for MNIST Inference 32
10.3 Measured Performance on ESP32 33
10.4 Exercises 34
Part 11: Performance Benchmarks 35
11.1 CNN1D Sine Model (1 FC Layer) 35
11.2 CNN1D Sine Model (2 FC Layers) 35
11.3 CNN2D MNIST on ESP32 35
11.4 Discussion 35
References 37

Part 1: Introduction to Neural Networks

1.1 What Is a Neural Network?

The term "neural" refers to neurons, and "network" describes how those neurons are connected. A neural network (NN) is a computational model inspired by the way biological neurons operate in the nervous system. In the brain, a neuron receives electrical signals through its dendrites, processes them in the cell body (soma), and sends the result along the axon to other neurons. Artificial neural networks mimic this pattern with a mathematical abstraction.

An artificial neuron receives several inputs, each multiplied by a corresponding weight. The weighted inputs are summed together with a bias term, and the result is passed through an activation function that introduces non-linearity. In mathematical form:

$$net = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

$$output = f(net)$$

where w_i are the weights, x_i the inputs, b the bias, and f the activation function.

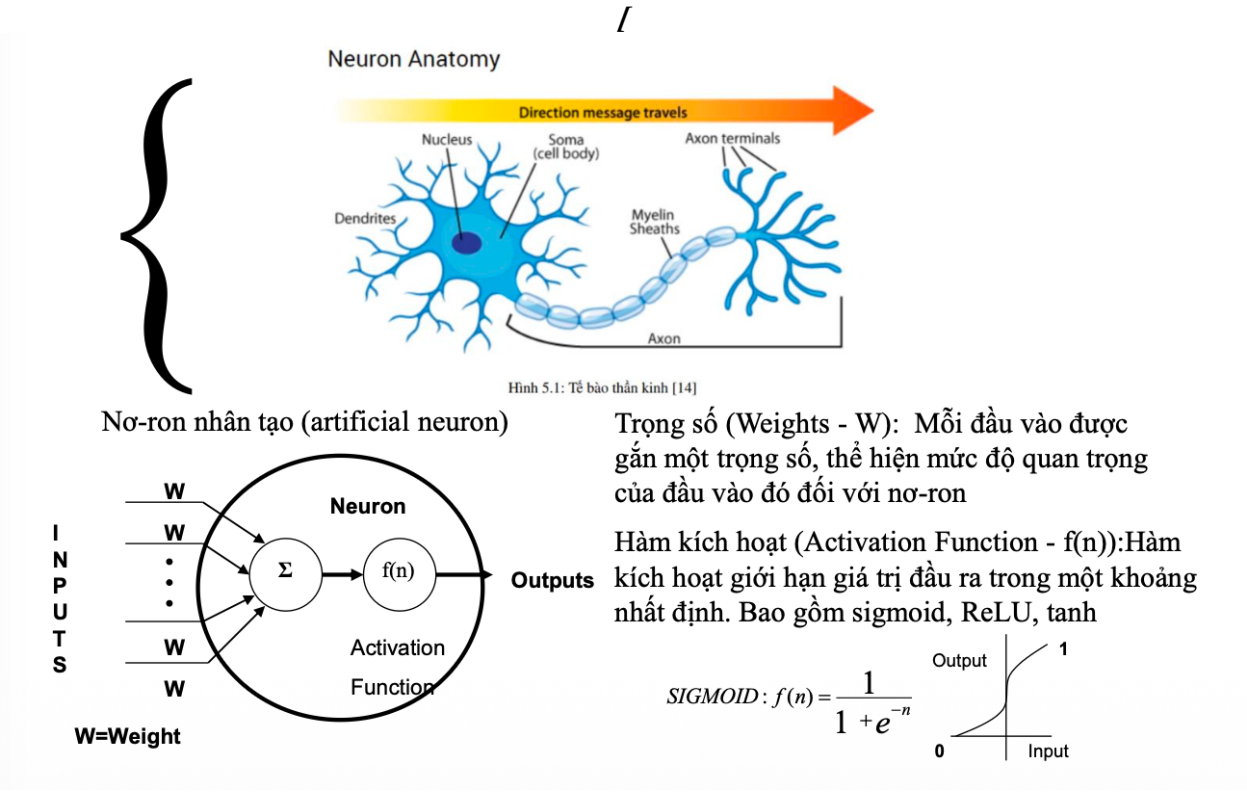
The components of an artificial neuron are:

- **Weights (W):** Each input is assigned a weight that reflects its relative importance. During training, these weights are adjusted through backpropagation to minimize prediction error.
- **Bias (b):** An additional learnable parameter that shifts the activation threshold. Without bias, the decision boundary would always pass through the origin.
- **Activation function f(n):** Constrains the output to a specific range and introduces the non-linearity that allows neural networks to approximate complex functions. Common choices include the Sigmoid function (outputs between 0 and 1), ReLU (outputs zero or the input itself), and Tanh (outputs between −1 and 1).

The Sigmoid function, for example, is defined as:

$$f(n) = 1 / (1 + e^{-n})$$

This S-shaped curve maps any real number to a value between 0 and 1, making it useful for modeling probabilities.

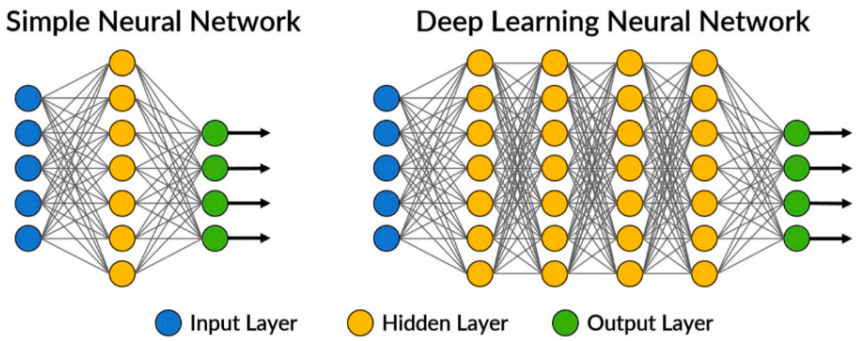


1.2 Simple Neural Networks versus Deep Neural Networks

A Deep Neural Network (DNN) is simply a neural network with multiple hidden layers between the input and output. Each layer performs non-linear transformations on its inputs, and deeper layers learn increasingly abstract representations of the data.

The practical differences between a shallow network and a deep one are significant:

Simple Neural Network	Deep Neural Network
Only 1 hidden layer	Multiple hidden layers (typically 3 or more)
Simple structure, fewer parameters	Complex structure, many more parameters
Suited for straightforward problems	Handles complex tasks like image recognition
Trains quickly	Requires more data and compute power
Limited feature learning capacity	Can learn hierarchical feature representations



Simple Neural Network

- Chỉ có 1 hidden layer
- Cấu trúc đơn giản, ít tham số
- Phù hợp với các bài toán đơn giản
- Tốc độ huấn luyện nhanh hơn
- Khả năng học đặc trưng bị giới hạn

Deep Learning Neural Network

- Có nhiều hidden layers
- Cấu trúc phức tạp, nhiều tham số hơn
- Xử lý được các bài toán phức tạp
- Khả năng học sâu các đặc trưng
- Cần nhiều dữ liệu và tài nguyên tính toán

1.3 How Training Works

Training a neural network is the process of finding the set of weights and biases that minimizes a chosen loss function. This happens iteratively through the following cycle:

First, in the forward pass, input data propagates through the network layer by layer, producing a prediction. Second, the loss function (such as Mean Squared Error for regression or Cross-Entropy for classification) measures how far the prediction is from the ground truth. Third, during backpropagation, the gradient of the loss with respect to every weight is computed using the chain rule of calculus. Finally, an optimizer such as Adam or SGD updates each weight by a small step in the direction that reduces the loss.

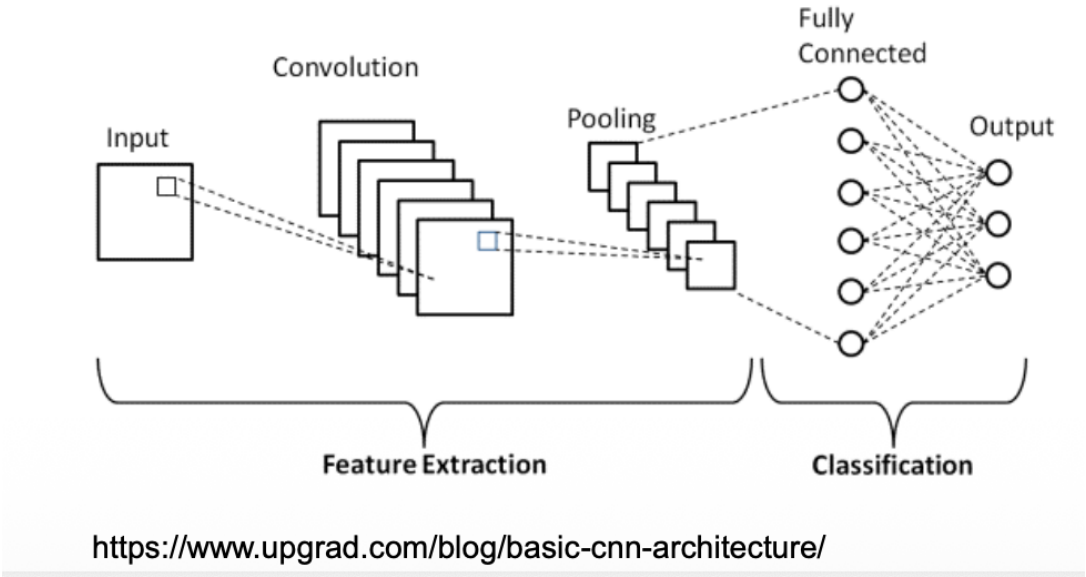
One complete pass through the entire training dataset is called an epoch. The batch size determines how many samples are used per weight update. A typical setup might use 20 epochs with a batch size of 32.

Part 2: Convolutional Neural Networks (CNN)

2.1 The Scaling Problem with Fully Connected Networks

Standard fully connected networks are impractical for spatial data like images. Consider a modest 32×32 pixel color image from the CIFAR dataset: a single neuron in the first hidden layer would need $32 \times 32 \times 3 = 3{,}072$ weights. For a 300×300 pixel image, that number explodes to 270,000 weights per neuron. Multiply that by hundreds of neurons, and the model becomes impossibly large to train.

Convolutional Neural Networks (CNNs) address this problem by replacing full connectivity with local, shared-weight filters. Instead of connecting every pixel to every neuron, a CNN slides a small filter (say, 3×3 pixels) across the image, computing a dot product at each position. Because the same filter weights are reused across the entire image, the number of parameters drops dramatically while the network retains the ability to detect spatial patterns.



2.2 The Convolutional Layer

The convolutional layer is the heart of a CNN. It applies a set of learnable filters (also called kernels) to the input. Each filter is a small matrix—commonly 3×3 or 5×5—that slides across the input and computes element-wise multiplications followed by a sum at every position. The result is a feature map (or activation map) that highlights where in the input a particular pattern was detected.

The important hyperparameters of this layer are:

- **Filter size (F):** The spatial dimensions of the kernel. A 3×3 filter covers a 3×3 region of the input at each step. Early layers often use small filters to detect edges and textures; some architectures use 1×1 filters for channel-wise mixing.

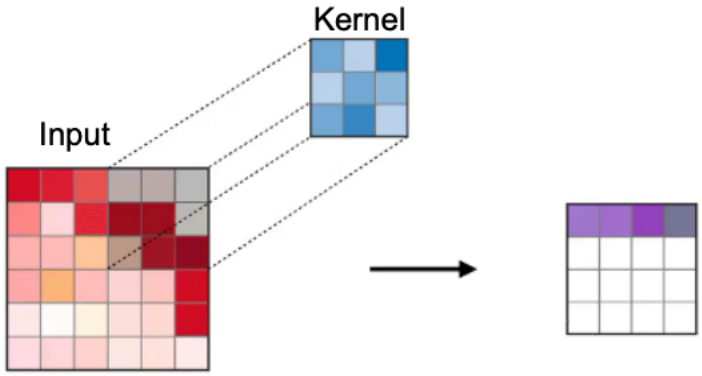
- **Stride (S):** How many pixels the filter moves between positions. A stride of 1 means the filter shifts one pixel at a time. Increasing the stride reduces the spatial size of the output.
- **Padding (P):** Zero-values added around the border of the input. With "same" padding, the output retains the same spatial dimensions as the input. With "valid" padding (no padding), the output shrinks.
- **Number of filters:** Determines how many different features this layer can detect. Each filter produces one feature map, so 32 filters yield 32 feature maps.

The output size of a convolutional layer is calculated as:

$$O = (I - F + 2P) / S + 1$$

where I is the input dimension, F the filter size, P the padding, and S the stride.

It is worth noting that Conv1D and Conv2D differ in their input dimensionality. Conv1D operates along a single axis, making it appropriate for time-series data and 1D signals. Conv2D operates across two spatial dimensions, making it the natural choice for images.

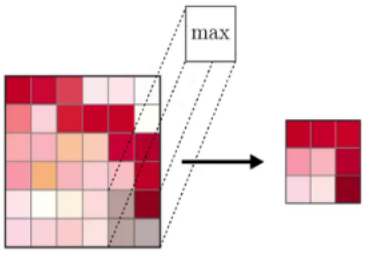
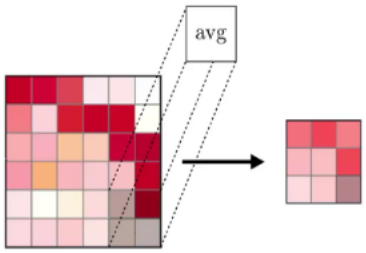


Source: https://stanford.edu/

2.3 The Pooling Layer

After convolution, pooling layers reduce the spatial dimensions of the feature maps. This serves two purposes: it makes the representation more compact (reducing computation in subsequent layers), and it provides a degree of translation invariance—the network becomes less sensitive to the exact position of a feature.

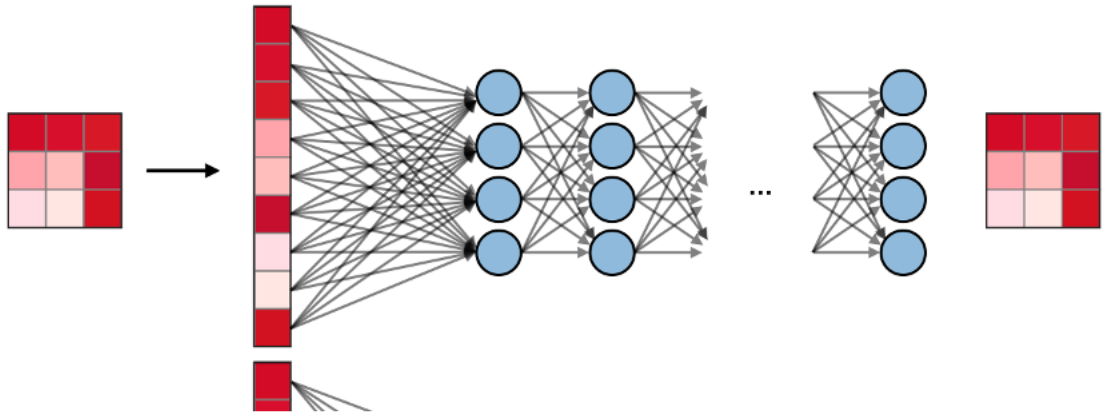
The two most common variants are Max Pooling, which keeps the largest value within each pooling window (preserving the strongest activations), and Average Pooling, which computes the mean value (producing smoother feature maps). In practice, Max Pooling with a 2×2 window and stride 2 is by far the most common choice, as it halves each spatial dimension while retaining the most prominent features.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used Phân loại đối tượng vì làm nổi bật đặc trưng	• Downsamples feature map • Used in LeNet Tác vụ yêu cầu làm mịn bản đồ đặc trưng (giảm nhiễu)

2.4 The Fully Connected Layer and Flatten

Once the convolutional and pooling layers have extracted spatial features, the resulting multi-dimensional tensor must be reshaped into a one-dimensional vector through a Flatten operation. For example, a tensor of shape 5×5×64 becomes a vector of 1,600 elements.

This flattened vector is then fed into one or more fully connected (FC) layers, where every element is connected to every neuron—just like in a traditional neural network. FC layers combine the extracted features and produce the final output, whether that is a set of class probabilities (for classification) or a continuous value (for regression).



Source: https://stanford.edu/

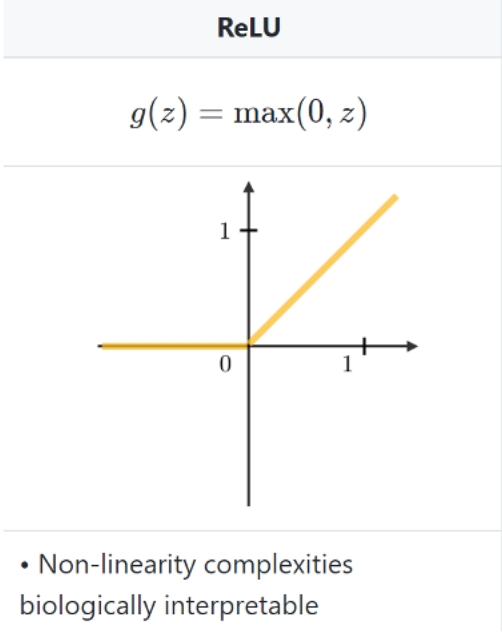
2.5 Activation Functions

2.5.1 ReLU

The Rectified Linear Unit (ReLU) is the dominant activation function in modern CNNs. It is defined as:

$$g(z) = max(0, z)$$

When z is negative, the output is zero; when z is non-negative, the output equals z. ReLU is computationally cheap (it requires only a comparison), it accelerates convergence during training, and it helps mitigate the vanishing gradient problem that plagues Sigmoid and Tanh in deep networks. One known issue is the "dying ReLU" problem, where neurons that output zero for all inputs stop learning. Variants such as Leaky ReLU (which allows a small slope for negative values) address this.



2.5.2 Softmax

For multi-class classification, the Softmax function is applied at the output layer. It takes a vector of raw scores (logits) and transforms them into a probability distribution:

$$p_i = exp(x_i) / \Sigma_j exp(x_j)$$

Each output p_i represents the probability that the input belongs to class i, and the sum of all probabilities equals 1. The predicted class is the one with the highest probability.

$$p = \begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix} \quad \text{where} \quad p_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Part 3: Running CNN on Microcontrollers

3.1 The Case for Edge AI

Microcontrollers (MCUs) such as the ESP32 are low-power, inexpensive devices commonly found in IoT applications. Traditionally, machine learning inference has been performed in the cloud, but this introduces network latency, requires a constant internet connection, and raises privacy concerns. Running CNN models directly on the microcontroller—an approach known as Edge AI or TinyML—eliminates these drawbacks.

When the CNN runs on-device, inference happens in real time without any server round-trip. Sensitive data like medical signals or voice recordings never leaves the device. The system works in remote locations without connectivity. And the power consumption stays in the milliwatt range, enabling battery-powered operation for extended periods.

The trade-off, of course, is that microcontrollers have severely limited resources compared to PCs or cloud GPUs. The ESP32, for example, has the following specifications:

Resource	ESP32 Specification	Practical Implication
CPU	Dual-core Xtensa LX6 at 240 MHz	Orders of magnitude slower than a GPU
SRAM	520 KB total (200–300 KB usable)	Must hold tensor arena and variables
Flash	4 MB (shared with firmware)	About 1–2 MB available for the model
FPU	Single-precision only	Float operations are slow; int8 quantization is essential

3.2 How Models Are Stored on ESP32

The model file (exported as a C header, typically with a .h extension) contains the entire model as a hexadecimal byte array. This array includes the network structure, all trained weights, and associated metadata. It is declared as a const array and stored in Flash memory (read-only program memory).

At runtime, the TensorFlow Lite Micro interpreter allocates a block of SRAM called the tensor arena. This arena holds all intermediate computation buffers, input tensors, and output tensors. The arena size must be set large enough to accommodate the largest intermediate tensor, plus some overhead. Finding the right arena size often requires experimentation: start large and gradually reduce until the model fails to allocate.

> **Note:** Flash holds the model permanently; SRAM holds temporary buffers. A model that is 100 KB in Flash but needs a 150 KB tensor arena consumes 250 KB of memory in total.

Part 4: TensorFlow Lite for Microcontrollers

4.1 Overview of TensorFlow Lite

TensorFlow Lite (TFLite) is the lightweight counterpart of TensorFlow, designed for inference on resource-constrained devices. The ecosystem includes three main components:

The Converter transforms a trained TensorFlow or Keras model into the .tflite FlatBuffer format, which is more compact and faster to parse than the original format.

The Interpreter runs .tflite models on mobile phones and single-board computers.

TFLite Micro (TFLM) is a stripped-down interpreter built specifically for bare-metal microcontrollers. It has a footprint of roughly 20 KB of code and does not rely on dynamic memory allocation or an operating system.

4.2 Optimization Techniques

Quantization is the most impactful optimization for microcontroller deployment. It converts 32-bit floating-point weights and activations to 8-bit integers, reducing the model size by approximately 4× and enabling faster computation on hardware without a strong floating-point unit. Two flavors exist: post-training quantization (simple, no retraining required) and quantization-aware training (more accurate, but the model must be retrained with simulated quantization).

Pruning removes weights whose values are close to zero, creating a sparser model. When combined with quantization, pruning can shrink a model by 10× or more with only a marginal drop in accuracy.

4.3 The Deployment Pipeline

The complete workflow from a trained model to a running inference on ESP32 involves five stages:

Step 1. Train the CNN model using TensorFlow and Keras on a PC with a suitable dataset.

Step 2. Convert the trained model to the .tflite format using the TFLite Converter, applying quantization.

Step 3. Export the .tflite file as a C byte array (a .h header file) so it can be compiled into the ESP32 firmware.

Step 4. Write the ESP32 firmware using Arduino IDE or ESP-IDF, linking against the TFLite Micro library.

Step 5. Flash the firmware to the ESP32 and evaluate performance: RAM usage, model size, inference time, and accuracy.

Part 5: Setting Up the Python Environment

5.1 Installing Dependencies

Python (version 3.8 or later) is the standard language for deep learning. Install the required packages in a virtual environment or with conda:

```
pip install tensorflow keras numpy matplotlib scikit-learn
```

Verify the installation:

```
import tensorflow as tf
print(tf.__version__)  # Expected: 2.x
```

5.2 Building a CNN with Keras

Keras, now integrated into TensorFlow 2.x, provides a high-level API for constructing and training neural networks. A sequential CNN model can be defined in just a few lines:

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=3,
        activation='relu', input_shape=(5, 1)),
    tf.keras.layers.MaxPooling1D(pool_size=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

In this example, Conv1D extracts 32 different features from the input using a kernel of size 3. MaxPooling1D downsamples by keeping the maximum value in each window of size 2. Flatten reshapes the output into a 1D vector. Dense(16) is a fully connected hidden layer with 16 neurons and ReLU activation. The final Dense(1) layer outputs a single value for regression.

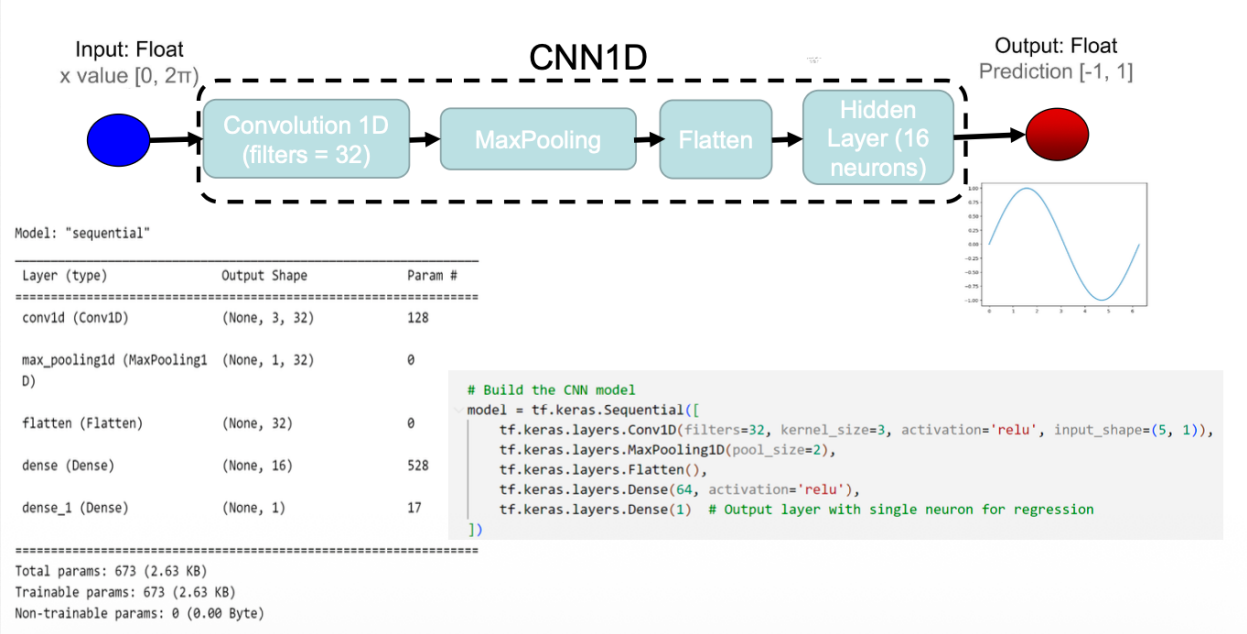
Part 6: Lab 1 — Sine Wave Prediction with CNN1D

6.1 Objective

In this lab, you will build a 1D convolutional neural network that learns to predict the sine function. The input is a set of phase values in the range [0, 2π], and the output is the corresponding sine value. Although the task is simple, it illustrates the complete pipeline: data preparation, model construction, training, evaluation, and visualization.

6.2 Architecture

The model consists of a Conv1D layer with 32 filters, followed by MaxPooling, Flatten, a Dense hidden layer with 16 neurons, and a single-neuron output. The total number of trainable parameters is 673 (2.63 KB)—small enough to fit comfortably in any microcontroller.



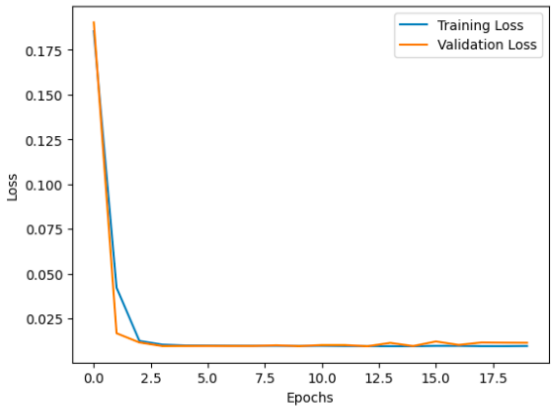

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d (Conv1D)             (None, 3, 32)             128

 max_pooling1d (MaxPooling1  (None, 1, 32)             0
 D)

 flatten (Flatten)           (None, 32)                0

 dense (Dense)               (None, 16)                528

 dense_1 (Dense)             (None, 1)                 17

=================================================================
Total params: 673 (2.63 KB)
Trainable params: 673 (2.63 KB)
Non-trainable params: 0 (0.00 Byte)
```

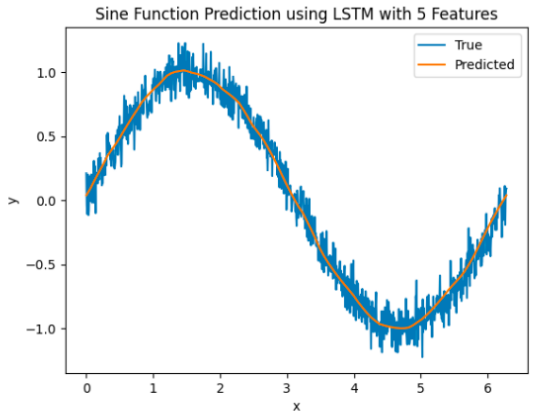
```python
# Build the CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=3, activation='relu', input_shape=(5, 1)),
    tf.keras.layers.MaxPooling1D(pool_size=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1)  # Output layer with single neuron for regression
])
```

6.3 Complete Code

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt


# Generate noisy sine data
np.random.seed(42)
x = np.linspace(0, 2*np.pi, 1000)
y = np.sin(x)
noise = np.random.normal(0, 0.05, x.shape)
y_noisy = np.sin(x + noise) + noise
```

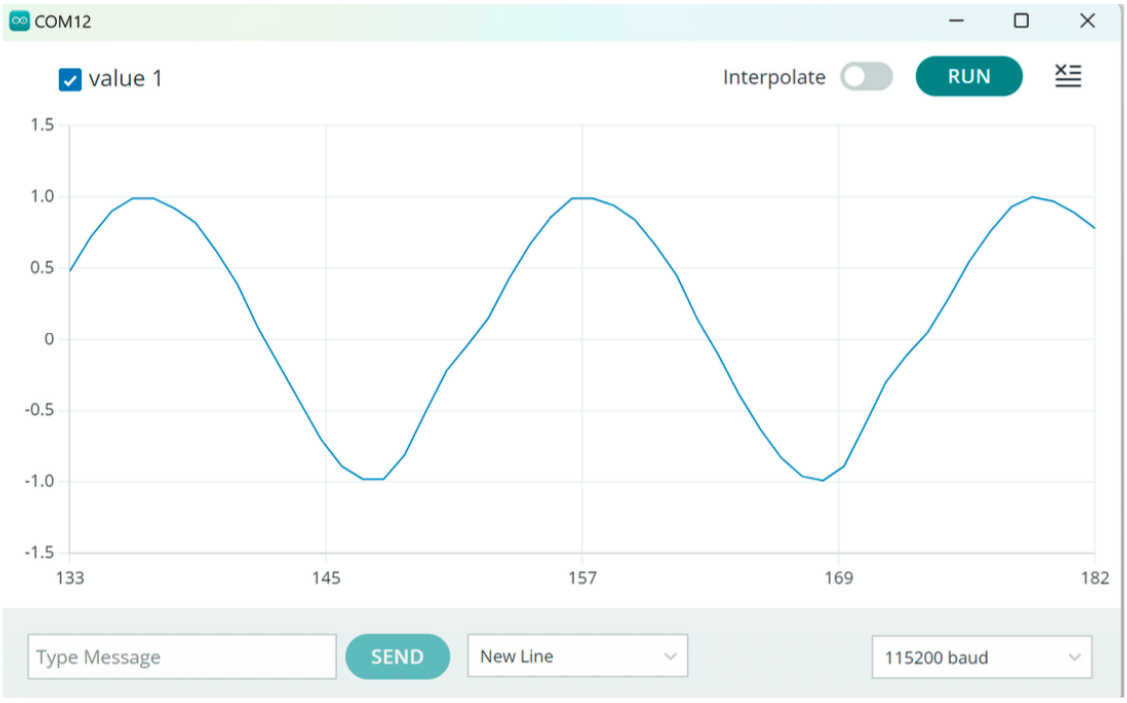
```
# Create sliding-window features
def create_features(data, window=5):
    return np.array([data[i:i+window]
                     for i in range(len(data) - window + 1)])

x_features = create_features(x).reshape(-1, 5, 1)
y_target = y_noisy[4:]  # align with feature windows

# Build model
model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(32, 3, activation='relu',
        input_shape=(5, 1)),
    tf.keras.layers.MaxPooling1D(2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
history = model.fit(x_features, y_target,
    epochs=20, batch_size=32, validation_split=0.2)

# Visualize results
y_pred = model.predict(x_features)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(history.history['loss'], label='Training Loss')
ax1.plot(history.history['val_loss'], label='Validation Loss')
ax1.legend(); ax1.set_title('Training Progress')

ax2.plot(x[4:], y[4:], label='True', alpha=0.7)
ax2.plot(x[4:], y_pred, label='Predicted', alpha=0.7)
ax2.legend(); ax2.set_title('Sine Prediction')
plt.tight_layout(); plt.show()

print(f'Validation MSE: {history.history["val_loss"][-1]:.4f}')
model.save('sin_cnn.h5')
```





val_mse: 0.0180

Hình SIN thu được từ mô hình CNN đã được đưa vào ESP32, được hiển thị trên Serial Plot.



6.4 Exercises

a) Run the code above and record the final validation MSE.

b) Change the number of Dense layer neurons to 32 and then 64. Build a comparison table of the MSE for each configuration.

c) Add a second Dense hidden layer and observe how it affects the MSE and training time.

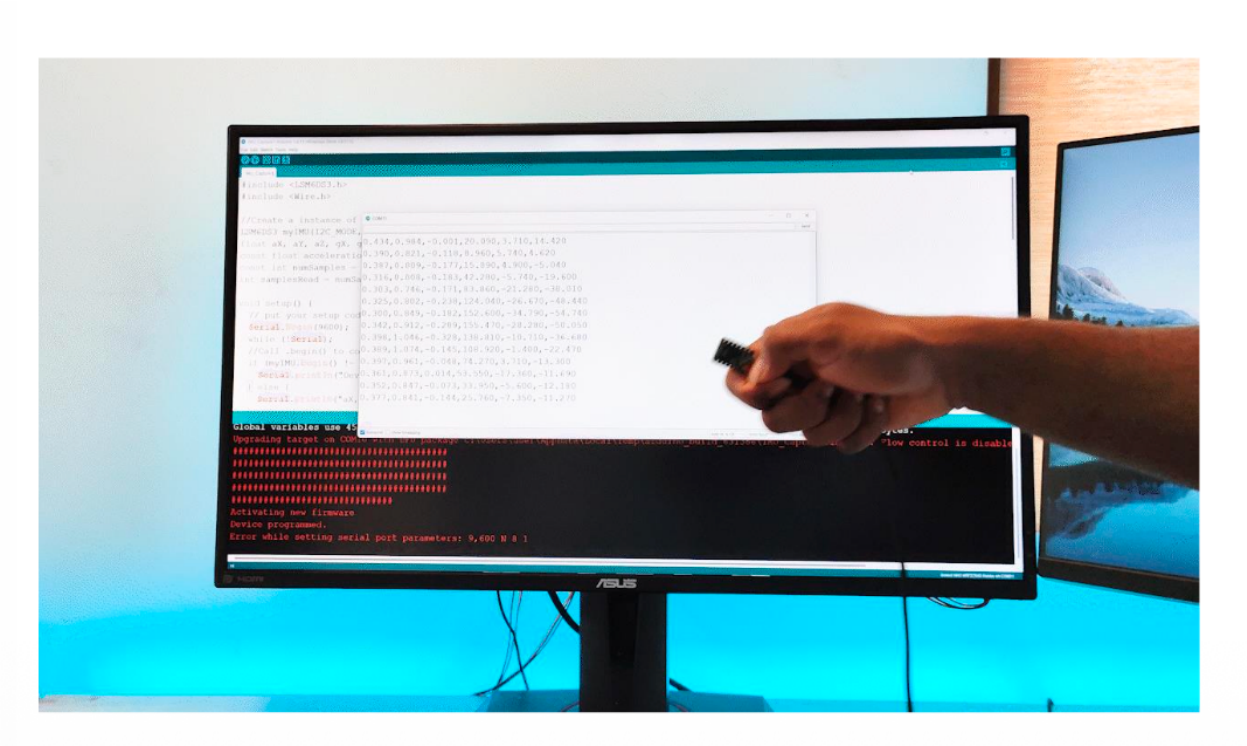
d) Plot the training loss and validation loss on the same graph for each configuration.

e) Save the trained model for use in Lab 4 (TFLite conversion).

Part 7: Lab 2 — Gesture Recognition with CNN1D and IMU

7.1 Objective

This lab extends the use of CNN1D to a real-world classification task: distinguishing between a punch gesture and a flex (arm curl) gesture using accelerometer and gyroscope data from an IMU sensor connected to an ESP32. The sensor provides six channels of data (aX, aY, aZ for acceleration and gX, gY, gZ for angular velocity), which together form a rich representation of hand movement.



7.2 Collecting Data

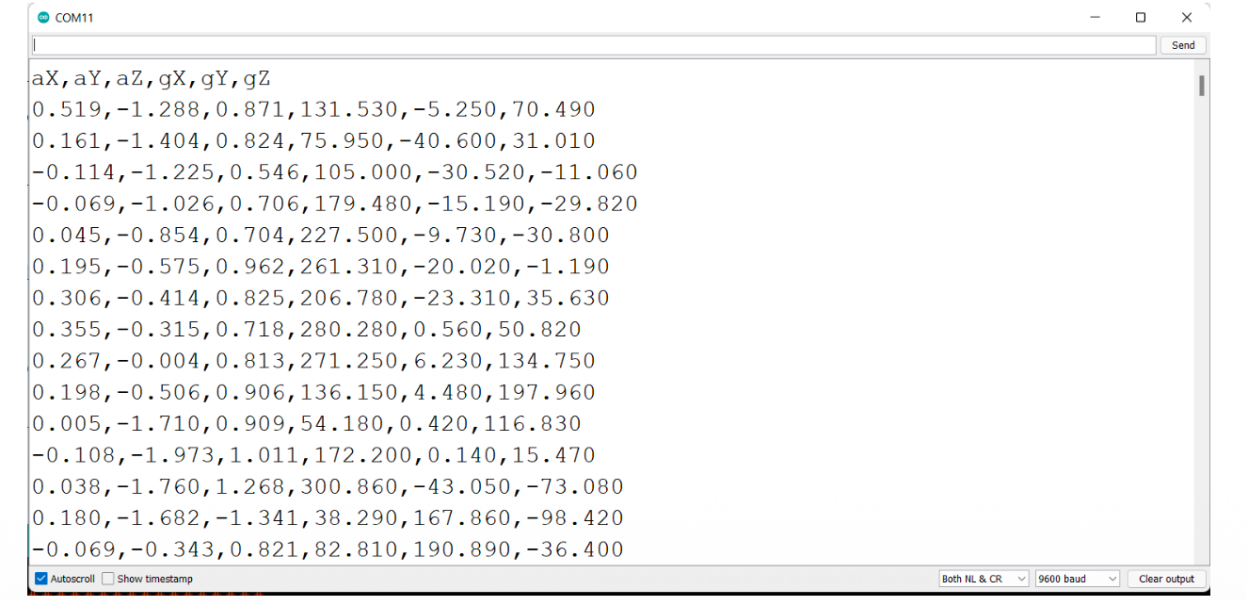
The IMU (such as the MPU6050) is connected to the ESP32 via I²C. A simple Arduino sketch reads the six-axis data at a fixed sampling rate and prints it over Serial in CSV format:

```
aX,aY,aZ,gX,gY,gZ
0.519,-1.288,0.871,131.530,-5.250,70.490
0.161,-1.404,0.824,75.950,-40.600,31.010
...
```

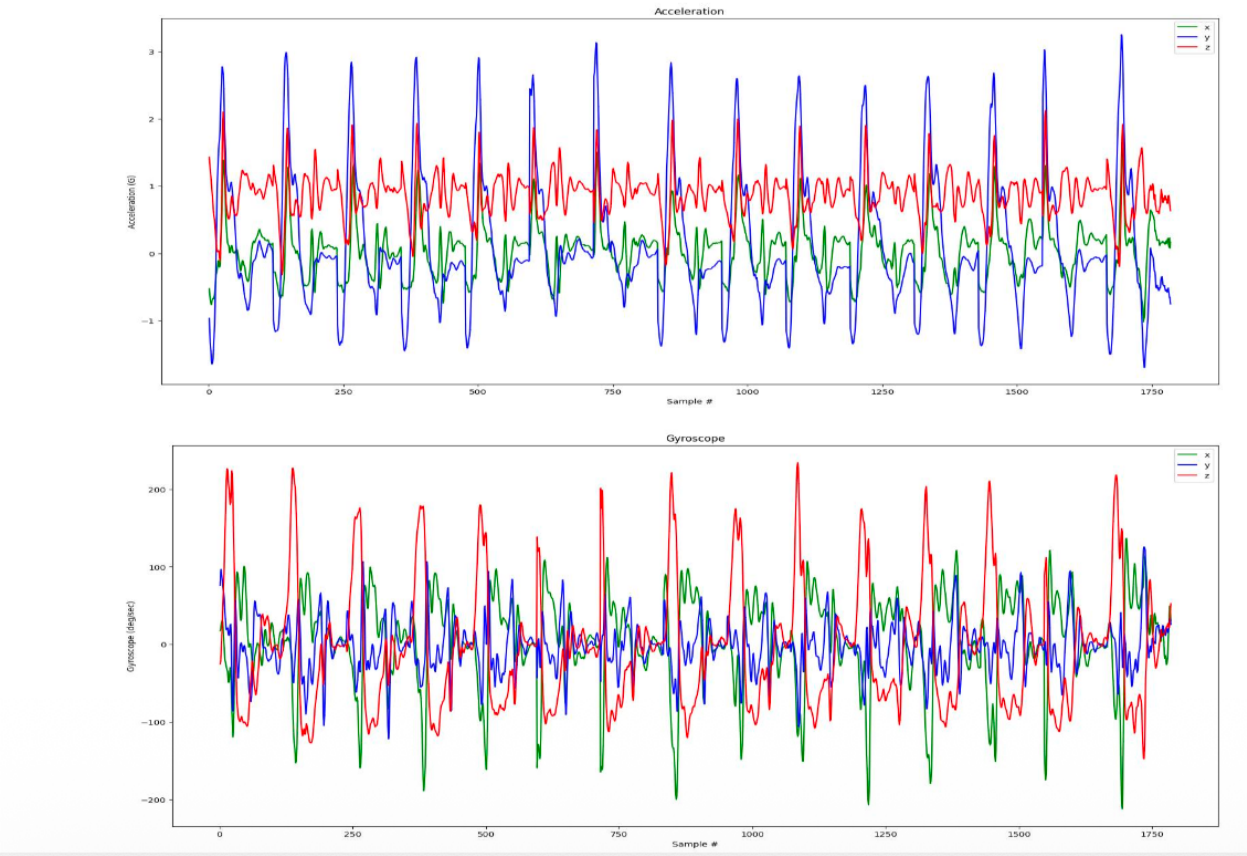
For each gesture class, perform 20 to 30 repetitions. Each repetition consists of approximately 119 samples at the default sampling rate. Save the data into separate files: punch.csv and flex.csv.

Thực hiện huấn luyện mô hình CNN phân biệt "**Hành động đấm và Co tay lại**" như trong hình sau:

Dữ liệu gia tốc kế và con quay hồi chuyển từ hành động đấm

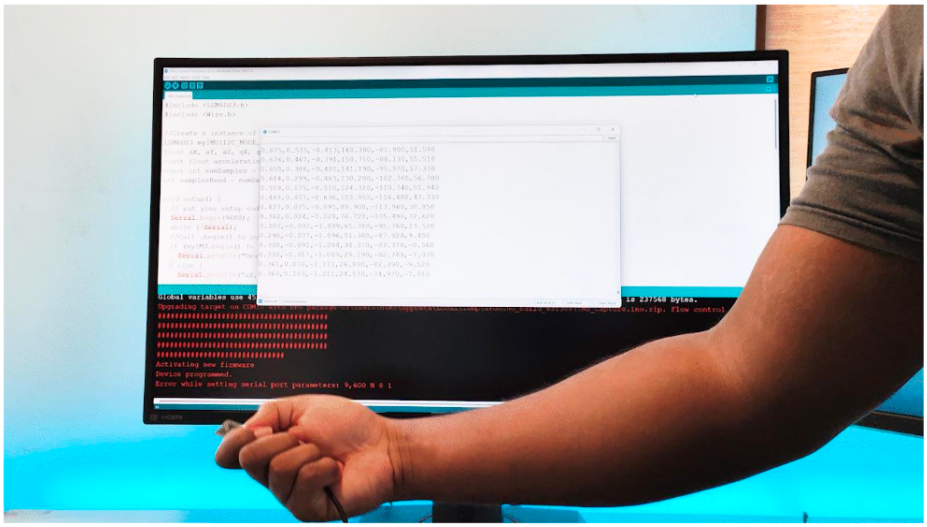


Vẽ Dữ liệu gia tốc kế và con quay hồi chuyển từ hành động đấm



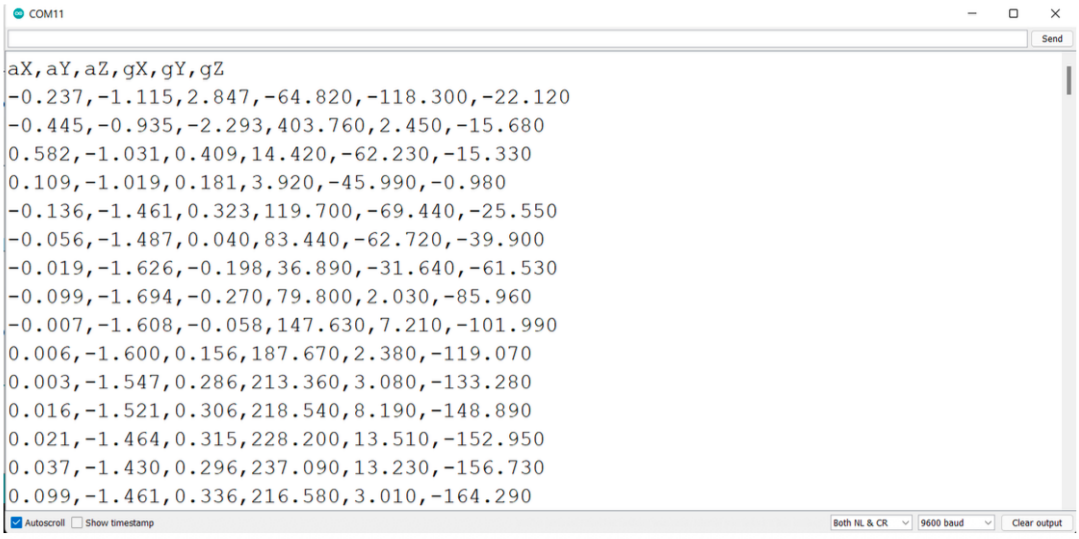
Thực hiện huấn luyện mô hình CNN phân biệt "**Hành động đấm và Co tay lại**" như trong hình sau:

Co tay lại



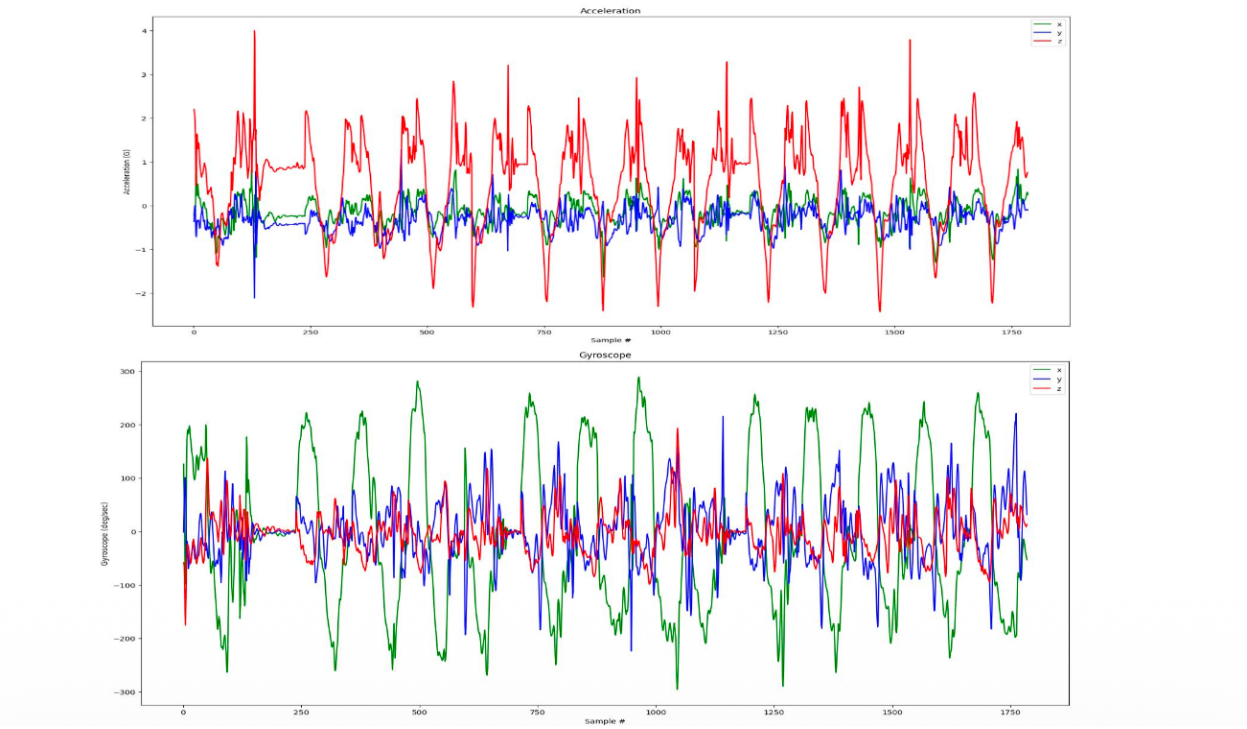
Thực hiện huấn luyện mô hình CNN phân biệt "**Hành động đấm và Co tay lại**" như trong hình sau:

Dữ liệu gia tốc kế và con quay hồi chuyển từ hành động co tay



Thực hiện huấn luyện mô hình CNN phân biệt "**Hành động đấm và Co tay lại**" như trong hình sau:

Vẽ Dữ liệu gia tốc kế và con quay hồi chuyển từ hành động co tay



7.3 Preprocessing and Model

```
import pandas as pd
import numpy as np
import tensorflow as tf

SAMPLES_PER_GESTURE = 119

def segment(df, label, n=119):
    segs, labs = [], []
    for i in range(0, len(df) - n + 1, n):
        segs.append(df.iloc[i:i+n].values)
        labs.append(label)
    return np.array(segs), np.array(labs)

punch = pd.read_csv('punch.csv')
flex  = pd.read_csv('flex.csv')

X_p, y_p = segment(punch, 0)
X_f, y_f = segment(flex,  1)
X = np.concatenate([X_p, X_f])
y = np.concatenate([y_p, y_f])

idx = np.random.permutation(len(X))
```

```
X, y = X[idx], y[idx]

model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(16, 3, activation='relu',
        input_shape=(119, 6)),
    tf.keras.layers.MaxPooling1D(2),
    tf.keras.layers.Conv1D(32, 3, activation='relu'),
    tf.keras.layers.MaxPooling1D(2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(2, activation='softmax')
])

model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

model.fit(X, y, epochs=30, batch_size=16,
    validation_split=0.2)
model.save('gesture_cnn.h5')
```

The model takes 119-timestep, 6-channel input and classifies it into one of two gesture categories. With adequate training data, accuracy above 95% is expected.

7.4 Exercises

a) Collect your own punch and flex data using ESP32 and MPU6050. Aim for at least 20 repetitions per gesture.

b) Train the model and report the validation accuracy and loss.

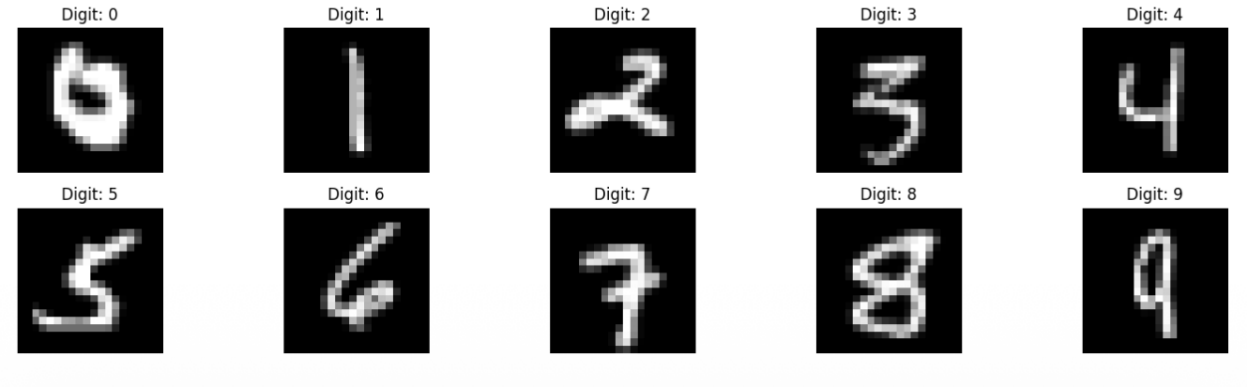
c) Add a third gesture (for example, a waving motion) and modify the output layer accordingly.

d) Convert the trained model to TFLite and deploy it on ESP32 using the procedure from Lab 4.

Part 8: Lab 3 — Handwritten Digit Recognition with CNN2D

8.1 Objective

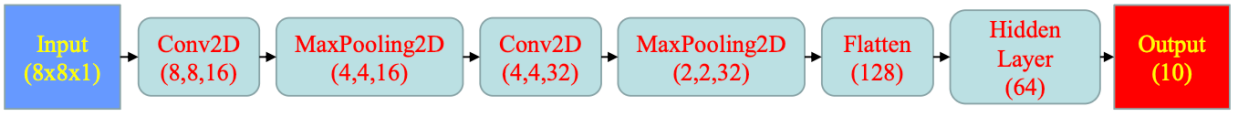
This lab tackles a classic image classification problem: recognizing handwritten digits from 0 to 9. To keep the model small enough for ESP32, we use the sklearn digits dataset, which consists of 1,797 images at 8×8 pixel resolution (grayscale). The dataset is split into approximately 90% training and 10% test samples.



8.2 Model Architecture

The CNN2D for 8×8 input follows this structure:

Input (8×8×1), then Conv2D with 16 filters of 3×3, "same" padding, and ReLU activation, followed by BatchNormalization, then MaxPooling2D with 2×2 window. This is repeated with 32 filters in the second convolutional block. After flattening the resulting 2×2×32 = 128 features, a Dense layer with 64 neurons and a final Dense layer with 10 neurons and Softmax activation produce the class probabilities.



Python Code:

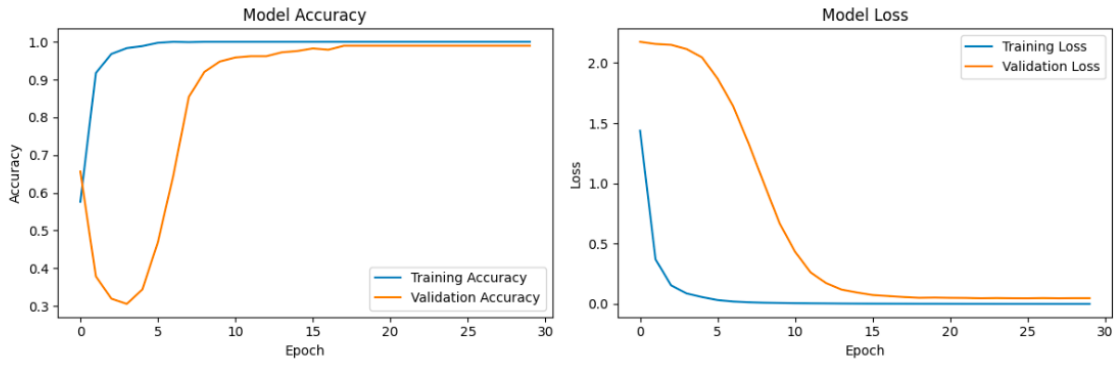
```
# Tạo model mới với kích thước input 8x8
self.model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(8, 8, 1)),
    tf.keras.layers.Conv2D(16, (3, 3), activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D((2, 2)),
    # tf.keras.layers.Dropout(0.25),

    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D((2, 2)),
    # tf.keras.layers.Dropout(0.25),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    # tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
```

8.3 Complete Code

```
import numpy as np
import tensorflow as tf
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Load and prepare data
digits = load_digits()
X = digits.data.reshape(-1, 8, 8, 1).astype('float32') / 16.0
y = digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42, stratify=y)

y_train_cat = tf.keras.utils.to_categorical(y_train, 10)
y_test_cat  = tf.keras.utils.to_categorical(y_test, 10)

# Build CNN2D model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(8, 8, 1)),
    tf.keras.layers.Conv2D(16, (3,3), activation='relu',
        padding='same'),
    tf.keras.layers.BatchNormalization(),
```

```
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Conv2D(32, (3,3), activation='relu',
       padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
    loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train_cat,
    epochs=30, batch_size=32, validation_split=0.1)

loss, acc = model.evaluate(X_test, y_test_cat)
print(f'Test accuracy: {acc:.4f}')

# Confusion matrix
y_pred = np.argmax(model.predict(X_test), axis=1)
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(
    cmap='Blues')
plt.title('Confusion Matrix'); plt.show()

model.save('mnist_8x8_cnn.h5')
```

Kết quả Huấn luyện:



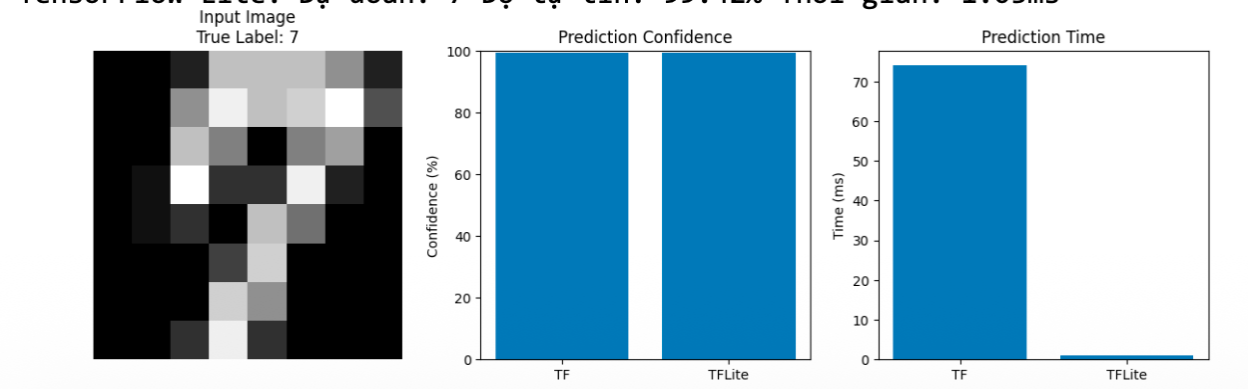
8.4 TensorFlow versus TFLite Performance

Framework	Accuracy	Avg. Inference Time
TensorFlow (on PC)	98.89%	71.46 ms
TensorFlow Lite (on PC)	98.89%	0.15 ms

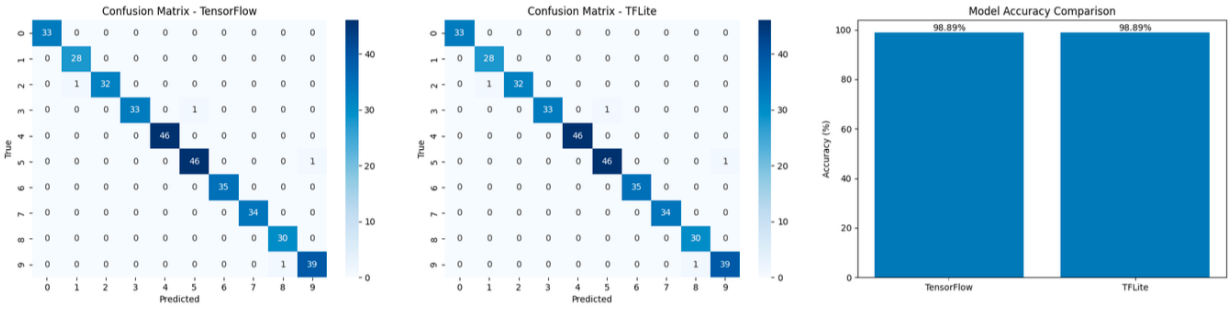
The TFLite model achieves identical accuracy while running roughly 470 times faster, a clear demonstration of the efficiency gains from optimization.

Kết quả đánh giá trên 1 mẫu trong tập Test:

```
Mẫu 4: Thực tế: 7
TensorFlow: Dự đoán: 7 Độ tự tin: 99.39% Thời gian: 74.04ms
TensorFlow Lite: Dự đoán: 7 Độ tự tin: 99.42% Thời gian: 1.03ms
```



Kết quả đánh giá trên toàn tập Test (180 mẫu):



	Độ chính xác	Thời gian thực thi (ms)
TensorFlow	98.89%	71.46
TensorFlow Lite	98.89%	0.15

8.5 Exercises

a) Run the full training pipeline and record the test accuracy.

b) Vary the number of Conv2D filters (try 8, 16, and 32) and the number of Dense neurons (32, 64, 128). Compare accuracy and model size for each.

c) Analyze the confusion matrix: which pairs of digits are most frequently confused?

d) Compare the .h5 and .tflite file sizes and compute the compression ratio.

Part 9: Lab 4 — Converting to TFLite and Exporting for ESP32

9.1 Conversion

```
import tensorflow as tf
import os

model = tf.keras.models.load_model('mnist_8x8_cnn.h5')

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

with open('mnist_model.tflite', 'wb') as f:
    f.write(tflite_model)

orig = os.path.getsize('mnist_8x8_cnn.h5') / 1024
lite = os.path.getsize('mnist_model.tflite') / 1024
print(f'Original: {orig:.1f} KB')
print(f'TFLite:   {lite:.1f} KB')
print(f'Reduction: {(1 - lite/orig)*100:.0f}%')
```

9.2 Verifying the TFLite Model

```
interpreter = tf.lite.Interpreter(
    model_path='mnist_model.tflite')
interpreter.allocate_tensors()

inp = interpreter.get_input_details()
out = interpreter.get_output_details()

correct = 0
for i in range(len(X_test)):
    interpreter.set_tensor(inp[0]['index'], X_test[i:i+1])
    interpreter.invoke()
    pred = np.argmax(
        interpreter.get_tensor(out[0]['index']))
    if pred == y_test[i]:
        correct += 1

print(f'TFLite accuracy: {correct/len(X_test)*100:.2f}%')
```

9.3 Exporting as a C Header

```
def export_c_header(tflite_path, header_path,
                    var='model_data'):
```

```
    with open(tflite_path, 'rb') as f:
        data = f.read()
    hex_str = ', '.join(f'0x{b:02x}' for b in data)
    with open(header_path, 'w') as f:
        f.write(f'// Auto-generated, {len(data)} bytes\n')
        f.write(f'const unsigned char {var}[] = ')
        f.write(f'{{{hex_str}}};\n')
        f.write(f'const int {var}_len = {len(data)};\n')
    print(f'Wrote {header_path}')

export_c_header('mnist_model.tflite', 'mnist_model_data.h')
```

9.4 Exercises

a) Convert each of the models from Labs 1, 2, and 3 to TFLite.

b) Record the original (.h5) and TFLite (.tflite) file sizes for each model.

c) Verify that the TFLite accuracy matches the original Keras model.

d) Generate .h header files ready for ESP32 integration.

Part 10: Lab 5 — Deploying the CNN on ESP32

10.1 Setting Up Arduino IDE

Step 1. Download and install Arduino IDE 2.0 or later.

Step 2. Open Preferences and add the ESP32 board manager URL:
https://dl.espressif.com/dl/package_esp32_index.json

Step 3. In Boards Manager, search for "esp32" and install the package.

Step 4. In Library Manager, search for "TensorFlowLite_ESP32" and install it.

Step 5. Select "ESP32 Dev Module" as the board and choose the correct COM port.

10.2 ESP32 Firmware for MNIST Inference

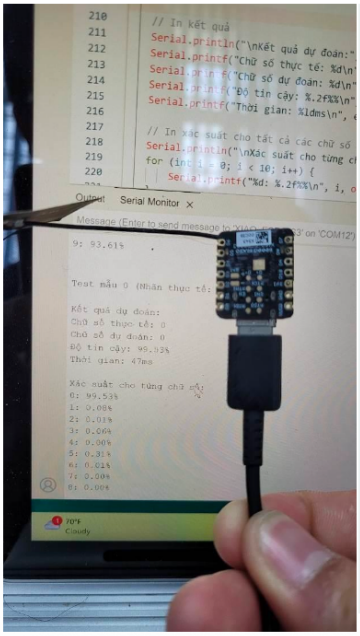
```
#include <TensorFlowLite_ESP32.h>
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "mnist_model_data.h"

constexpr int kArenaSize = 150 * 1024;
uint8_t tensor_arena[kArenaSize];

tflite::MicroInterpreter* interpreter = nullptr;
TfLiteTensor* input  = nullptr;
TfLiteTensor* output = nullptr;

// A sample digit "0" (8x8, normalized to 0..1)
const float test_digit[] = {
  0.00, 0.00, 0.31, 0.87, 0.94, 0.56, 0.00, 0.00,
  0.00, 0.19, 0.94, 0.69, 0.50, 1.00, 0.19, 0.00,
  0.00, 0.38, 0.94, 0.00, 0.00, 0.81, 0.50, 0.00,
  0.00, 0.38, 0.94, 0.00, 0.00, 0.63, 0.63, 0.00,
  0.00, 0.31, 0.94, 0.00, 0.00, 0.63, 0.56, 0.00,
  0.00, 0.19, 1.00, 0.13, 0.00, 0.75, 0.44, 0.00,
  0.00, 0.06, 0.88, 0.75, 0.75, 0.94, 0.13, 0.00,
  0.00, 0.00, 0.31, 0.75, 0.81, 0.38, 0.00, 0.00
};

void setup() {
  Serial.begin(115200);
  while (!Serial) delay(10);

  const tflite::Model* model =
      tflite::GetModel(model_data);

  static tflite::AllOpsResolver resolver;
  static tflite::MicroInterpreter interp(
```

```
    model, resolver, tensor_arena, kArenaSize);
  interpreter = &interp;
  interpreter->AllocateTensors();

  input  = interpreter->input(0);
  output = interpreter->output(0);

  Serial.printf("Arena used: %d bytes\n",
    interpreter->arena_used_bytes());
}

void loop() {
  // Load test image into input tensor
  for (int i = 0; i < 64; i++)
    input->data.f[i] = test_digit[i];

  unsigned long t0 = millis();
  interpreter->Invoke();
  unsigned long dt = millis() - t0;

  // Find the class with highest probability
  int best = 0;
  float best_p = output->data.f[0];
  for (int i = 1; i < 10; i++) {
    if (output->data.f[i] > best_p) {
      best_p = output->data.f[i];
      best = i;
    }
  }

  Serial.printf("Predicted: %d  Confidence: %.2f%%  Time: %ldms\n", best, best_p*100, dt);

  // Print all probabilities
  for (int i = 0; i < 10; i++)
    Serial.printf("  %d: %.2f%%\n",
      i, output->data.f[i]*100);

  delay(5000);
}
```

10.3 Measured Performance on ESP32

Metric	Measured Value	Notes
RAM (tensor arena)	145 KB	Out of ~300 KB available
Model size in Flash	366 KB	Stored as const array
Inference time	47 ms	Per single 8×8 image
Prediction confidence	99.53%	For test digit “0”

Đánh giá kết quả của mô hình CNN2D trên ESP32 để dự đoán mẫu Test số “0”. Kết quả dự đoán đúng số “0” với độ tin cậy 99.53% và thời gian thự thi 47ms. Trong khi đó, độ Lớn model chiếm 366kB và được nạp vào Bộ nhớ Flash và Ram của ESP32 chiếm 145 KB, như hình mô tả sau:



```
Test mẫu 0 (Nhãn thực tế: 0)

Kết quả dự đoán:
Chữ số thực tế: 0
Chữ số dự đoán: 0
Độ tin cậy: 99.53%
Thời gian: 47ms

Xác suất cho từng chữ số:
0: 99.53%
1: 0.08%
2: 0.01%
3: 0.06%
4: 0.00%
5: 0.31%
6: 0.01%
7: 0.00%
8: 0.00%
9: 0.00%
```

CNN2D	64 Neurons
RAM	145 KB
Model size	366 KB
Execution Time	47 ms

10.4 Exercises

a) Flash the firmware and confirm that the prediction matches the expected digit.

b) Swap in a different test digit array and verify the model handles it correctly.

c) Record the arena_used_bytes printed by the firmware. Try decreasing kArenaSize and find the smallest value that still works.

d) Deploy the sine model from Lab 1 on the same ESP32 and compare the RAM usage and inference time to the MNIST model.

Part 11: Performance Benchmarks

11.1 CNN1D Sine Model (1 FC Layer)

1 FC Layer	16 Neurons	32 Neurons	64 Neurons
RAM	95.8 KB	98 KB	102 KB
Model Size	38 KB	52 KB	79 KB
Exec. Time	385 µs	415 µs	453 µs
MSE	0.018	0.036	0.013

11.2 CNN1D Sine Model (2 FC Layers)

2 FC Layers	16 Neurons	32 Neurons	64 Neurons
RAM	97.4 KB	98 KB	119 KB
Model Size	48 KB	52 KB	183 KB
Exec. Time	407 µs	460 µs	620 µs
MSE	0.01	0.01	0.0091

11.3 CNN2D MNIST on ESP32

CNN2D MNIST	TF (PC)	TFLite (PC)	ESP32
Accuracy	98.89%	98.89%	99.53%*
Inference Time	71.46 ms	0.15 ms	47 ms
RAM	—	—	145 KB
Model Size	~1.5 MB	~366 KB	366 KB

* Confidence on a single test sample; the overall test-set accuracy is 98.89%.

11.4 Discussion

Several patterns emerge from the benchmark data. First, adding neurons consistently increases RAM usage, model size, and inference time, but it does not always improve accuracy. The 32-neuron single-layer model actually performed worse (MSE 0.036) than the 16-neuron variant (MSE 0.018), likely due to overfitting on this small dataset. Second, adding a second FC layer improved the MSE from 0.018 to 0.01 for 16 neurons, but the additional cost in RAM and time was modest. Third, the CNN2D MNIST model shows that even a relatively complex 2D classification task can run comfortably on ESP32 within 47 ms per inference, well within the requirements of most real-time applications.

The overarching lesson is that bigger models are not necessarily better on constrained hardware. Careful architecture design, combined with quantization, yields models that are both accurate and efficient.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems, vol. 25, 2012.

[3] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

[4] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation, 2016, pp. 265–283.

[5] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018.

[6] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in Advances in Neural Information Processing Systems, vol. 28, 2015.

[7] C. R. Banbury et al., "Benchmarking TinyML systems: Challenges and direction," arXiv preprint arXiv:2003.04821, 2021.

[8] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA: O'Reilly Media, 2019.

[9] Stanford CS230 CNN Cheatsheet. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/

[10] TensorFlow Lite for Microcontrollers. [Online]. Available: https://www.tensorflow.org/lite/microcontrollers

[11] Scikit-learn Digits Dataset. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

[12] Espressif, ESP32 Technical Reference Manual. [Online]. Available: https://www.espressif.com/en/products/socs/esp32